

A Hardware Accelerator for Adaptive Dormand-Prince RK45 ODE Solvers

Daniel Strapp

Electrical and Computer Engineering
University of Toronto
Toronto, Canada

daniel.strapp@mail.utoronto.ca

Harry Le

Electrical and Computer Engineering
University of Toronto
Toronto, Canada

harry.le@mail.utoronto.ca

ABSTRACT

We present a fully pipelined, double-precision (IEEE 754 FP64) implementation of the Dormand-Prince RK45 adaptive ODE solver on a Xilinx FPGA. Existing FPGA-based ODE solvers typically suffer from three compounding limitations: fixed step sizes, single-precision arithmetic, and hardcoded system equations. This design addresses all three simultaneously using pipelined multiply-accumulations, overlapped error computation, and a microcode-based engine for runtime ODE reconfiguration without FPGA reprogramming. Validated against a SciPy RK45 reference on a moderately stiff ODE, the solver achieves a maximum absolute error of 6.82×10^{-7} across 43 adaptive steps, a 3–4 order-of-magnitude improvement over a fixed-step FP32 RK4 baseline at comparable step count, and completes integration up to $6.06\times$ faster than SciPy on a desktop CPU.

CCS CONCEPTS

- Hardware~Integrated circuits~Reconfigurable logic and FPGAs~Hardware accelerators

KEYWORDS

FPGA accelerator, adaptive Runge-Kutta, Dormand-Prince RK45, ordinary differential equations, Verilog, Xilinx Artix-7

1 Introduction

Ordinary differential equations (ODEs) are used to represent models across physics, chemistry, control theory, and embedded simulation. Unfortunately, most real-world ODEs lack closed-form analytical solutions, requiring numerical integration. When high accuracy is required or the ODE is stiff (characterized by widely separated eigenvalues that force a small step size for stability), adaptive step-size integration algorithms are standard practice. The Dormand-Prince embedded pair [1], which underlies MATLAB's widely used 'ode45' routine [2], computes a 5th-order solution and a free 4th-order error estimate from the same seven function evaluations. This error estimate drives an adaptive controller that adjusts step size h to satisfy a user-prescribed tolerance, avoiding both over-computing on smooth regions and instability on stiff ones.

Software implementations of such methods introduce overhead from loop control, branch misprediction, and memory bandwidth pressure that can be prohibitive in latency-critical embedded applications like real-time plant simulation, closed-loop control, or parameter estimation. FPGAs offer fixed-function datapaths that execute Runge-Kutta (RK) multiply-accumulate chains with deterministic low latency. Prior work on FPGA-based ODE acceleration has demonstrated significant throughput gains [3, 4, 5, 6], but all implementations target fixed-step methods. The adaptive step feedback loop has received comparatively limited hardware attention.

Bhattacharya and Chakraborty [7] demonstrated hardware accelerators for RK2, RK3, and RK4 ODE solvers using single-precision (FP32) Xilinx floating-point IP cores on a Zynq-ZC702 FPGA, with a custom RISC-V ISA interface for runtime control. Their work establishes the viability of FPGA-based ODE acceleration and provides detailed power, resource, and timing analysis across solver orders. However, three limitations constrain its applicability: (1) all solvers use a fixed step size, relying on the user to choose h appropriately; (2) arithmetic is single-precision, introducing truncation errors that accumulate over longer integration intervals; and (3) the ODE function $f = x^2 - y^2$ is hardcoded in RTL, requiring re-synthesis for any other problem. This work directly extends [7] by addressing all three limitations. The contributions are:

1. Adaptive RK45 in hardware: A fully pipelined, double-precision Dormand-Prince RK45 solver with accept/reject step control, safety-factor scaling, and overshoot prevention, implemented in synthesizable SystemVerilog targeting Xilinx FPGAs.
2. Programmable ODE engine: A microcode-based function evaluator supporting up to 16 arithmetic instructions and 6 user constants, allowing any ODE to be loaded at runtime without re-synthesis.
3. Quantitative validation: Functional verification against a scipy RK45 software reference and comparison with the fixed-step RK4 (FP32) baseline from [7], demonstrating the accuracy, adaptive behavior, and speedup of the design on a moderately stiff test problem.

2 Architecture and Design

2.1 System Overview

Figure 1 shows the top-level structure. The five major submodules are:

- FSM Controller (Figure 3): orchestrates the integration loop by issuing step requests to the RK45 step engine, latching accept/reject decisions from the step controller, writing accepted (x, y, err) triples to the output buffer, and advancing x until the integration endpoint is reached
- RK45 Step Engine (Figure 2): computes stages k_1 – k_7 and error estimate via a shared RK45 stage instance
- Step Controller (Figure 4): adaptive step-size scaling and overshoot prevention
- Output Buffer: 256×192-bit FIFO for accepted (x, y, err) triples
- Support Units: for x advancement and interval termination

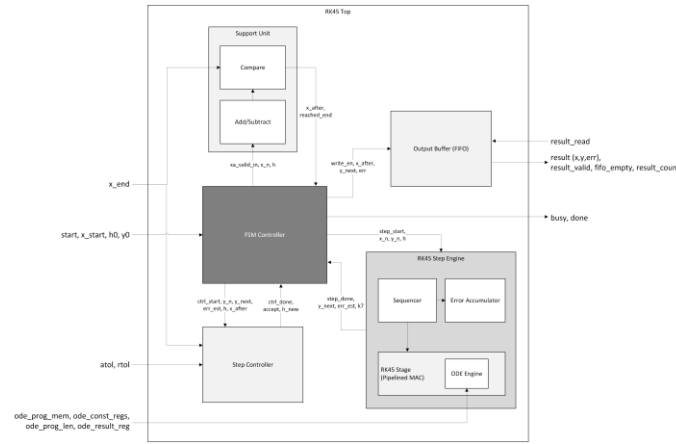


Figure 1: Top level block diagram of RK45

2.2 Floating-Point Subsystem

All arithmetic uses IEEE 754 double-precision (64-bit) via Xilinx Floating Point (FP) v7.1 IP cores [8]: add/subtract (11 cycles), multiply (6 cycles), divide (28 cycles), and compare-GT (2 cycles). Double precision is required because adaptive error estimates are sensitive to truncation; FP32 can cause false step rejections or missed error growth on stiff problems.

2.3 Pipelined Stage Computation

Each stage computes a weighted sum of prior stage derivatives using Butcher tableau coefficients [9], then evaluates the ODE function. The RK45 Step Engine (Figure 2) feeds a_{ij}/k_j pairs into the 6-cycle multiplicative pipeline one per cycle, buffers products as they emerge, and accumulates them sequentially via add/subtract (11 cycles each). Once accumulation completes, the multiplications $h \times \text{accum}$ and $x_n + c \cdot h$ execute in parallel on separate FP units. The stage then calls the ODE Engine to produce k_s .



Figure 2: RK45 Step Engine

2.4 Overlapped Error Accumulation

A key parallelization optimization exploits the observation that after stage k_6 completes, the error accumulation for stages k_0 through k_5 is independent of k_7 . The RK45 Step Engine therefore launches k_7 and the error accumulation in parallel (Figure 2): dedicated units work concurrently with the ~ 130 -cycle k_7 pipeline, hiding ~ 100 cycles of error work. Once both finish, the k_7 contribution is folded in and $\text{err_est} = h \cdot \Sigma(e_i \cdot k_i)$ is finalized.

2.5 Adaptive Step Controller

The adaptive step controller implements standard Dormand-Prince error norm and scaling (Figure 4):

$$e_{\text{norm}} = \frac{|e_{\text{est}}|}{a_{\text{tol}} + r_{\text{tol}} \cdot \max(|y_n|, |y_{\text{next}}|)}$$

The step is accepted if $e_{\text{norm}} \leq 1$ and rejected otherwise, with the next step size calculated as:

$$h_{\text{new}} = h \cdot \text{clamp}(0.9 \cdot e_{\text{norm}}^{-0.2}, 0.2, 10)$$

The exponent $e^{-0.2}$ is computed by a custom FP module using Newton-Raphson iterations [10]. A final guard clamps h_{new} to $x_{\text{end}} - x_{\text{current}}$ to prevent overshoot.

2.6 Programmable ODE Engine

The ODE engine executes user-defined ODEs from a 16-entry instruction memory (ADD/SUB/MUL/DIV) with an 8×64-bit register file ($R0=x$, $R1=y$, $R2$ – $R7$ user constants). The test function $f(x, y) = -50(y - x) + 1$ compiles to three instructions. The fetch-execute-writeback loop adds 28 cycles per evaluation but eliminates re-synthesis for new ODEs.

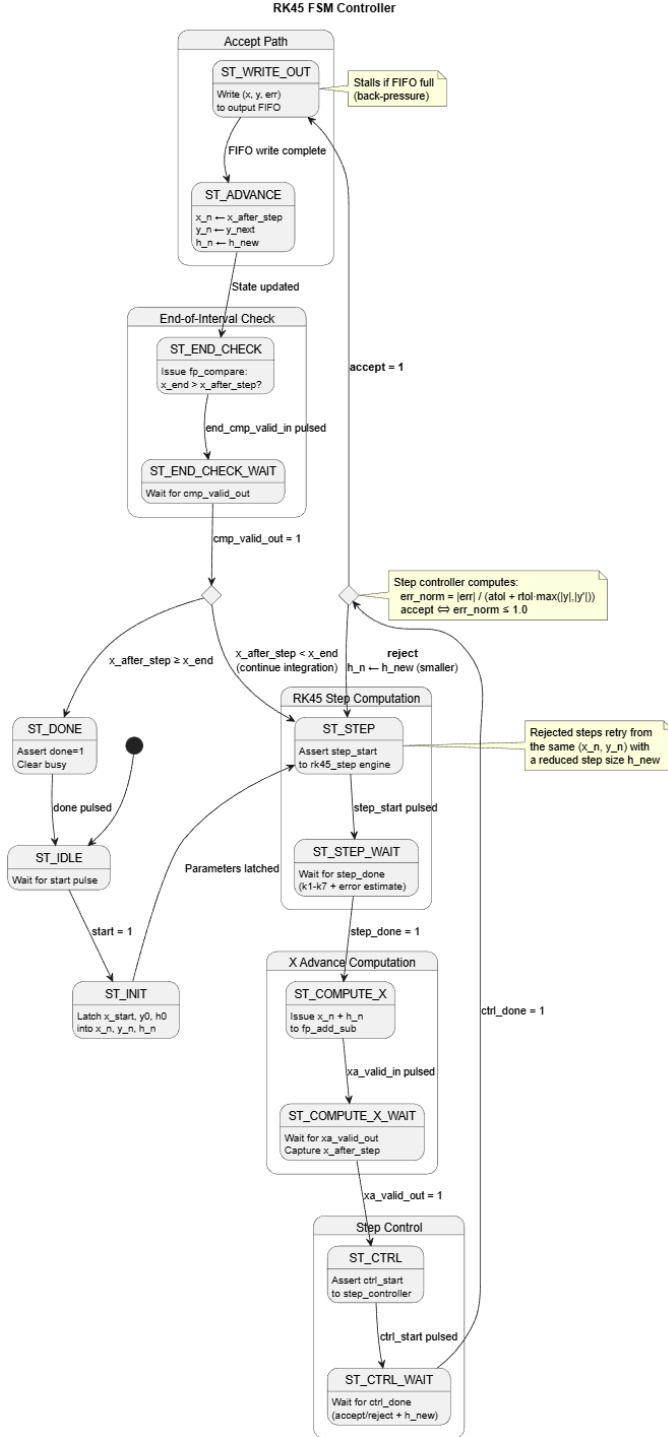


Figure 3: FSM Controller State Diagram

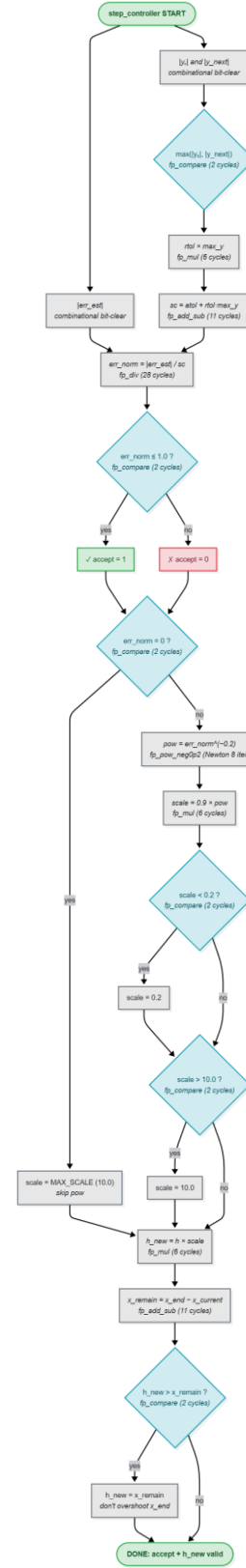


Figure 4: Adaptive step-size selection flowchart

3 Methodology

3.1 Test Problem

The design is validated using the ODE:

$$\frac{dy}{dx} = -50(y - x) + 1, y(0) = 1, x \in [0, 1]$$

with exact analytical solution $y(x) = x + e^{-50x}$. This is a moderately stiff problem: the exponential transient decays rapidly near $x = 0$, requiring small step sizes, while the solution smooths to near-linear behavior for larger x . This makes it an effective test of the adaptive step-size controller's ability to concentrate computational effort where needed.

3.2 Simulation Environment

Functional verification used Vivado xsim at 100 MHz (10 ns). The 100 MHz clock was chosen to match the baseline in [7] for a fair resource and power comparison. Solver settings were $rtol = 10^{-6}$, $atol = 10^{-9}$, and $h_0 = 10^{-3}$.

3.3 Validation Strategy

Checks were performed at three levels:

1. Unit level: The testbench verifies FP primitives against known values (tolerance 10^{-12}).
2. System level: The RK45 testbench runs the 3-instruction ODE program and records 43 accepted samples (x, y, err) , compared against $y_{exact} = x + e^{-50x}$.
3. Reference comparison: Results are compared against scipy `solve_ivp(method='RK45')` and the fixed-step FP32 RK4 baseline from [7] ($h = 0.025$, 40 steps).

3.4 Limitations

Results are reported for one scalar ODE and one tolerance setting; broader generalization to stiff, oscillatory, or vector systems requires additional benchmarks. Hardware metrics are post-synthesis estimates only; post-route timing/power and board measurements are still needed for final sign-off. Only the test ODE $f = -50(y - x) + 1$ was exercised; generalization to multi-term or nonlinear $f(x, y)$ is architecturally supported by the microcode engine but has not been verified in simulation.

4 Results

4.1 Accuracy of the Hardware RK45 Solver

The hardware solver completes integration of the test ODE over $x \in [0, 1]$ in 43 accepted steps. Table 1 summarizes the error metrics, computed against the exact analytical solution $y(x) = x + e^{-50x}$.

Metric	Value
Max absolute error	6.82×10^{-7} ($x \approx 0.770$)
Mean absolute error	1.23×10^{-7}
Final absolute error ($x = 1$)	8.26×10^{-8}

Table 1: Hardware RK45 accuracy summary

All 43 steps satisfy the error norm $e_norm \leq 1.0$, confirming the adaptive controller never exceeds the prescribed tolerance ($rtol = 10^{-6}$, $atol = 10^{-9}$) at any accepted point. Errors are largest near $x \approx 0.77$, where the exponential transient has fully decayed and accumulated rounding in the near-linear $y \approx x$ regime is the dominant error source.

4.2 Improvement Over Fixed-Step RK4

Figure 5 compares the hardware RK45 solver against the fixed-step FP32 RK4 baseline from [7] ($h = 0.025$, 40 uniform steps). Table 2 summarizes the accuracy comparison.

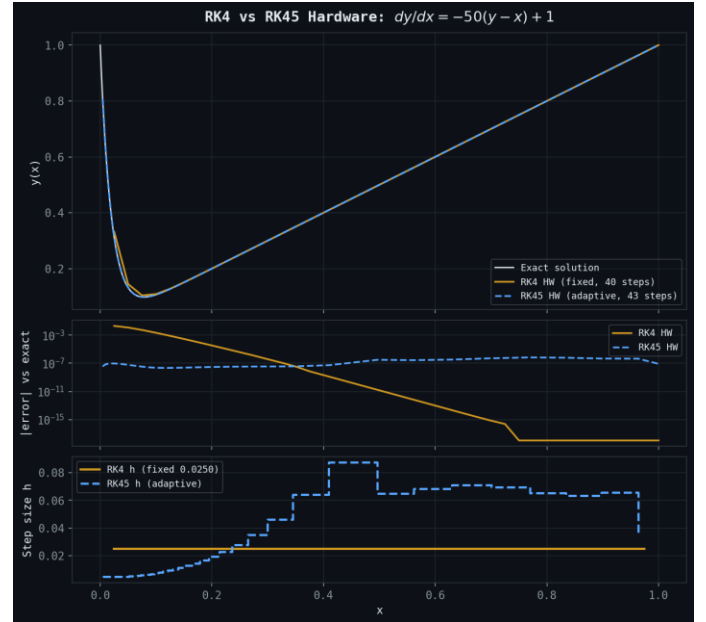


Figure 5: Hardware RK45 vs Hardware RK4 solving the test problem

Metric	HW RK45 (FP64, adaptive)	HW RK4 (FP32, fixed $h = 0.025$)
Steps	43	40
Max absolute error	6.82×10^{-7} ($x \approx 0.770$)	2.10×10^{-2} ($x \approx 0.025$)
Mean absolute error	1.23×10^{-7}	1.06×10^{-3}
Final absolute error ($x = 1$)	8.26×10^{-8}	4.17×10^{-7}

Table 2: Hardware RK45 (FP64, adaptive) vs Hardware RK4 (FP32, fixed step) accuracy comparison

RK45 achieves a 3–4 order-of-magnitude reduction in max and mean absolute error with a comparable number of steps. The gain comes from two complementary sources: the adaptive controller concentrates steps in the stiff initial transient where the fixed $h = 0.025$ is too coarse, while FP64 arithmetic avoids the truncation error that accumulates in FP32 over the full interval.

4.3 Improvement over Software RK45

Table 3 compares the hardware implementation against `scipy solve_ivp(method='RK45')` running the same ODE with identical tolerances ($\text{rtol} = 10^{-6}$, $\text{atol} = 10^{-9}$) and integration bounds.

Metric	Hardware RK45 (FP64)	scipy RK45 (FP64)
Accepted steps	43	43
Max absolute error	6.82×10^{-7} ($x \approx 0.770$)	8.07×10^{-7} ($x \approx 0.776$)
Mean absolute error	1.23×10^{-7}	1.13×10^{-7}
Final absolute error ($x = 1$)	8.26×10^{-8}	7.70×10^{-8}
Step-size range	$4.38 \times 10^{-3} - 8.73 \times 10^{-2}$	$4.62 \times 10^{-3} - 8.82 \times 10^{-2}$
Steps with $h \leq 0.010$	23	23
Steps with $0.010 < h \leq 0.030$	8	8
Steps with $h > 0.030$	12	12
Wall clock (HW @ 100 MHz)	655.8 μs	2,144 μs
Wall clock (HW @ Fmax)	354 μs	2,144 μs

Table 3: Hardware RK45 vs. scipy RK45 accuracy and timing comparison

Both solvers complete integration in exactly 43 accepted steps with matching per-region step counts and peak step sizes within 1% of each other (8.73×10^{-2} vs. 8.82×10^{-2}). Hardware and scipy absolute errors agree to within 20% at the peak and within 7% at the final point $x = 1$, confirming that the hardware controller faithfully replicates the adaptive behaviour of the reference algorithm. The small step-placement differences visible in Figure 6 are attributable to independent floating-point rounding in the step-size scaling formula and do not reflect systematic bias.

The hardware solver completes the 43-step integration in 65,580 cycles (655.8 μs at 100 MHz; $\sim 354 \mu\text{s}$ projected at Fmax (Section 4.4)), compared to 2,144 μs for scipy RK45 on an Intel i9-9900 at 3.1 GHz, yielding speedups of $3.27\times$ and $6.06\times$ respectively. These gains are conservative: scipy benefits from a mature optimized runtime, and the hardware platform is a low-end Artix-7.

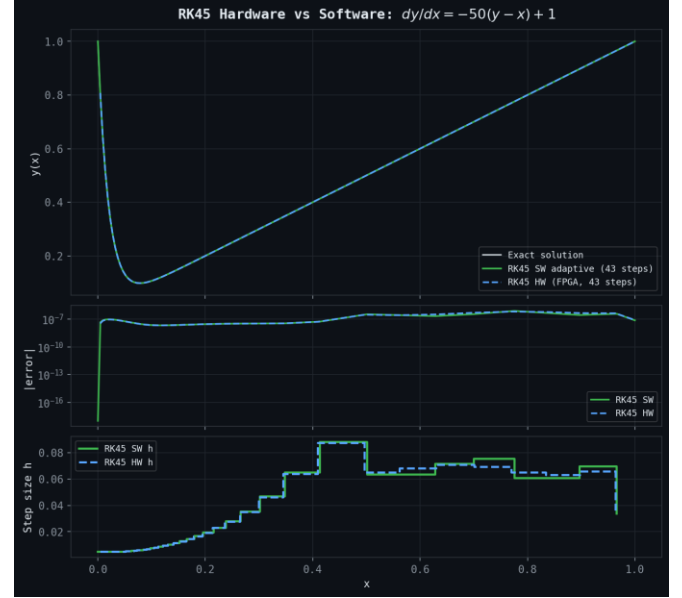


Figure 6: Hardware RK45 vs scipy RK45 solving the test problem

4.4 Synthesis Results

Figure 7 and Table 4 compare post-synthesis resource utilization between the hardware RK45 design and the RK4 baseline from [7], both targeting the Artix-7 xc7a200t at 100 MHz.

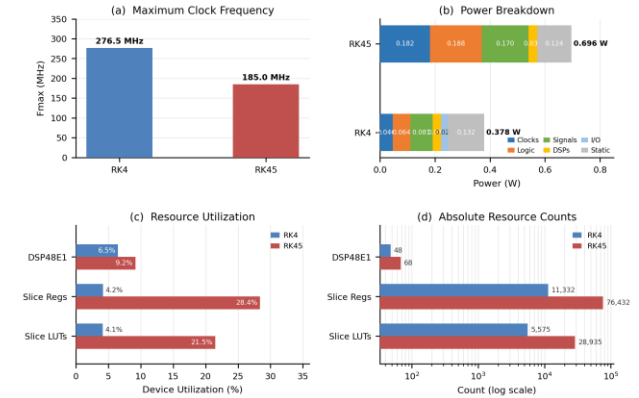


Figure 7: Post-synthesis resource utilization comparison

Resource	HW RK45 (FP64, adaptive)	HW RK4 (FP32, fixed) [7]
Slice LUTs	28,935 (21.5%)	5,575 (4.14%)
Slice Registers	76,432 (28.4%)	11,332 (4.21%)
DSP48E1	68 (9.2%)	48 (6.49%)
Fmax	≈ 185 MHz (WNS = 4.593 ns)	≈ 276 MHz (WNS = 6.383 ns)
On-chip Power	0.696 W	0.378 W

Table 4: Post-synthesis resource utilization data

The higher area and power of the RK45 design relative to the RK4 baseline are expected: FP64 operators are wider than their FP32 equivalents, and the design adds adaptive step-size control, error estimation, and the microcoded ODE engine. The high flip-flop count (76 K) reflects the deep pipeline registers required to sustain the 6–28 cycle FP64 latencies across all concurrently active pipeline stages. The design uses no block RAM; all pipeline state and Butcher tableau coefficients are held in flip-flops and shift-register LUTs.

4.5 Discussion

The results support the central claim of this work: FPGA-based adaptive RK45 with FP64 arithmetic is both practical and significantly more accurate than fixed-step FP32 RK4 at comparable step count. The 3–4 order-of-magnitude accuracy improvement (Table 2) arises from adaptive step concentration in the stiff transient and the avoidance of FP32 truncation error over the full interval. The near-exact match with scipy RK45 (Table 3) validates that the hardware FSM and step controller correctly implement the Dormand-Prince algorithm, with the micro-coded ODE engine providing runtime flexibility without re-synthesis. The $3.27\text{--}6.06\times$ speedup over scipy on a desktop CPU (Table 3) further demonstrates that this accuracy is not achieved at a latency cost, making the design viable for real-time applications where both precision and throughput are required.

5 Related Work

Early work by Shirazi et al. [3] demonstrated that pipelined floating-point arithmetic on FPGAs could accelerate computationally intensive algorithms, laying the foundation for dedicated numerical solvers. Building on that foundation, several groups implemented fixed-step ODE accelerators. Chen et al. [4] demonstrated FPGA-accelerated simulation of physical-model ODEs for real-time control. Stamoulas et al. [5] developed hardware accelerators supporting both FP32 and FP64 precision and performed explorations across performance and resource metrics. Huang et al. [6] introduced the Differential Equation Processing Element (DEPE), a custom FPGA processor with accompanying compilation tools for mapping physical-model ODE systems to hardware. Fedoseev et al. [11] investigated parallelization strategies for Runge-Kutta pipelines on FPGA, showing that exploiting pipeline concurrency reduces per-step latency significantly.

The most direct prior work is Bhattacharya and Chakraborty [7], who implemented RK2, RK3, and RK4 accelerators using Xilinx FPU IP cores [8] on a Zynq-ZC702, providing detailed comparative analysis across solver orders. Hollabough and Chakraborty [12] subsequently demonstrated a programmable co-processor for Lotka-Volterra equations, showing that FPGA ODE engines can be made problem-configurable, though still using a fixed integration step.

Across all prior work, two limitations remain consistent: the step size is fixed, requiring users to choose h conservatively for stiff

problems, and the ODE function evaluation is either hardcoded in RTL or constrained to a specific problem class. No prior hardware implementation has combined adaptive step-size control with a runtime-programmable ODE evaluator in a fully pipelined FP64 datapath. This work addresses that gap directly.

6 Future Directions

Several extensions would strengthen and generalize this work:

1. Implementation closure: place-and-route plus board-level measurement for final timing, utilization, and power.
2. First Same as Last (FSAL) optimization: reuse k_7 as next-step k_1 to eliminate one of seven stage evaluations per accepted step (~ 130 cycles saved).
3. Stage-level parallelism: instantiate multiple RK45 stage engines to trade area for latency, following the approach of Fedoseev et al. [11].
4. Vector ODE and scalability: extend the datapath to systems of ODEs with a vector error norm, enabling physical-model simulation workloads as in [6].
5. Implicit methods: hardware implementations of implicit RK or backward differentiation formula (BDF) solvers (necessary for strongly stiff problems) remain an open problem across all prior work.

REFERENCES

- [1] J. R. Dormand and P. J. Prince, "A family of embedded Runge-Kutta formulae," *Journal of Computational and Applied Mathematics*, vol. 6, no. 1, pp. 19–26, Mar. 1980.
- [2] L. F. Shampine and M. W. Reichelt, "The MATLAB ODE suite," *SIAM Journal on Scientific Computing*, vol. 18, no. 1, pp. 1–22, 1997.
- [3] N. Shirazi, A. Walters, and P. Athanas, "Quantitative analysis of floating point arithmetic on FPGA-based custom computing machines," in *Proc. IEEE FCCM*, 1995, pp. 155–162.
- [4] H. Chen, S. Sun, and D. C. Aliprantis, "Dynamic simulation of DFIG wind turbines on FPGA boards," in *Proc. IEEE Power and Energy Society General Meeting*, 2010, pp. 1–8. doi:10.1109/PES.2010.5437160
- [5] I. Stamoulas, M. Möller, R. Miedema, and C. Strydis, "High-performance hardware accelerators for solving ordinary differential equations," in *Proc. Int. Workshop on Highly Efficient Accelerators and Reconfigurable Technologies*, ACM, 2017.
- [6] C. Huang, F. Vahid, and T. Givargis, "A custom FPGA processor for physical model ordinary differential equation solving," *IEEE Embedded Systems Letters*, vol. 3, no. 4, pp. 89–92, 2011.
- [7] S. Bhattacharya and D. Chakraborty, "Design and Analysis of a Hardware Accelerator with FPU-Based Runge-Kutta Solvers," in *Proc. 2023 Int. Conf. on Electrical, Communication and Computer Engineering (ICECCE)*, 2023, doi:10.1109/ICECCE61019.2023.10442325.
- [8] AMD/Xilinx, *Floating-Point Operator v7.1 Product Guide (PG060)*, 2021.
- [9] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 2nd ed. John Wiley & Sons, 2008.
- [10] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Cengage Learning, 2011.
- [11] P. Fedoseev, K. Zhukov, D. Kaplun, and N. Vybornov, "Parallelization of Runge-Kutta methods for hardware implementation," *Computation*, vol. 10, no. 12, p. 215, 2022.
- [12] A. Hollabough and D. Chakraborty, "An open-source co-processor for solving Lotka-Volterra equations," in *Proc. 2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2022, doi:10.1109/ISVLSI54635.2022.00094.